

Classificadores Bagging, Boosting e Support Vector Machine

Cesar de Oliveira F. Silva

Setembro / 2026

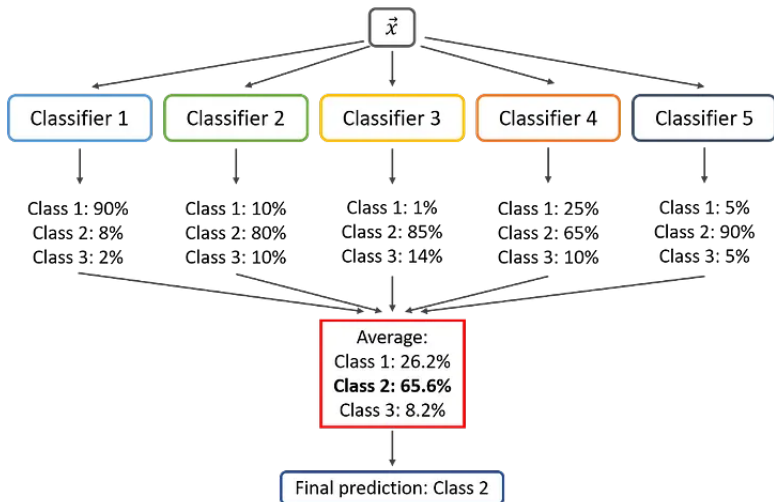
Roteiro

- 1 Motivação
- 2 Ensembles
- 3 Support Vector Machines (SVM)
- 4 Classificação Multiclasse e kNN

Por que usar comitês de classificadores?

- **Ideia central:** em vez de ter um único modelo “sabe-tudo”, combinamos vários modelos mais simples.
- Analogia: pedir a opinião de **vários especialistas** e fazer uma votação.
- Vantagens:
 - Reduzir erros de um modelo individual.
 - Aumentar robustez a ruído e variações do conjunto de treino.
 - Explorar diferentes visões dos dados (diferentes atributos, amostras, algoritmos).
- Esses métodos são chamados de **ensembles** ou **comitês de classificadores**.

Comitês de Classificadores (Ensemble Classifiers)



Elementos de um comitê

- **Classificadores individuais** (membros do comitê):
 - Podem ser do mesmo tipo (várias árvores) ou de tipos diferentes.
 - Podem usar subconjuntos distintos de atributos e/ou amostras.
- **Mecanismo de combinação:**
 - Votação simples (classe mais votada).
 - Média das probabilidades ou das saídas (regressão).
 - Combinações ponderadas (classificadores mais confiáveis recebem peso maior).
- Objetivo: produzir uma **predição composta** melhor que cada modelo isolado.

Arquiteturas de comitês

- **Arquitetura paralela**

- Todos os modelos recebem a mesma entrada.
- As saídas são combinadas ao final (votação ou média).
- Ex.: Bagging, Random Forest.

- **Arquitetura em cascata**

- Modelos são executados em sequência (pipeline).
- Cada estágio pode descartar classes ou refinar a decisão.
- Normalmente, classificadores simples/rápidos primeiro; complexos depois.

- **Arquitetura hierárquica**

- Estrutura parecida com uma árvore de decisão.
- Nem todos os classificadores são usados para todas as amostras.

- **Arquiteturas híbridas:** combinações de paralela, cascata e hierárquica.

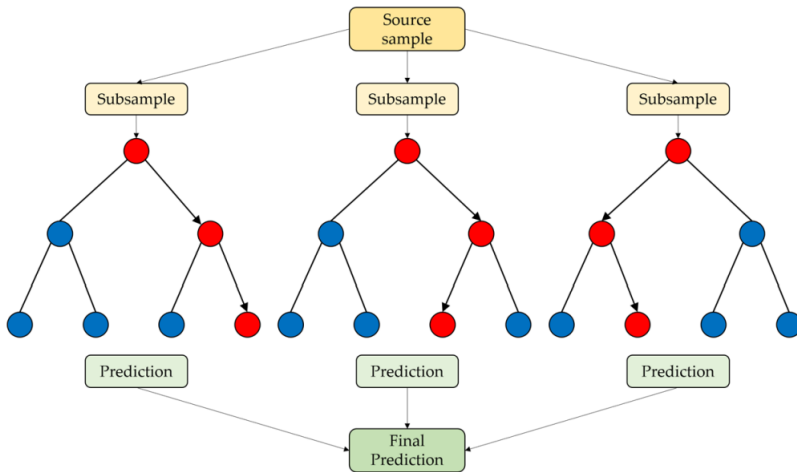
Ensembles



Bagging: Bootstrap Aggregating

- **Bagging** = *Bootstrap aggregating*.
- Objetivo: reduzir a **variância** de modelos instáveis (ex.: árvores de decisão).
- Ideia:
 - 1 A partir do conjunto de treino original, gerar M amostras **bootstrap** (reestratos com reposição).
 - 2 Treinar um modelo em cada amostra.
 - 3 Combinar as predições (votação ou média).
- Analogia: vários estudantes resolvendo a mesma prova a partir de listas de exercícios ligeiramente diferentes; depois fazemos uma votação.

Bagging



Por que o Bagging funciona?

- Suponha M classificadores base, cada um com taxa de erro $t < 0,5$.
- Se os erros fossem **independentes**, a probabilidade da maioria errar seria bem menor que t .
- Na prática, os modelos não são totalmente independentes, mas o Bagging ainda:
 - Reduz a variância (o modelo fica menos sensível a pequenas mudanças nos dados).
 - Mantém o viés aproximadamente igual.
- Regra prática: bagging é especialmente útil para modelos **de alta variância** (árvores profundas, por exemplo).

Random Forest

- **Random Forest** = Bagging aplicado a árvores de decisão + aleatorização extra.
- Duas fontes de aleatoriedade:
 - **Bootstrap** das amostras (como no Bagging).
 - **Subconjunto aleatório de atributos** em cada nó: só um conjunto de m atributos é considerado para dividir o nó.
- Benefícios:
 - Árvores mais diversificadas $\Rightarrow$ ensemble mais forte.
 - Boa performance “de saída” com pouco ajuste fino.

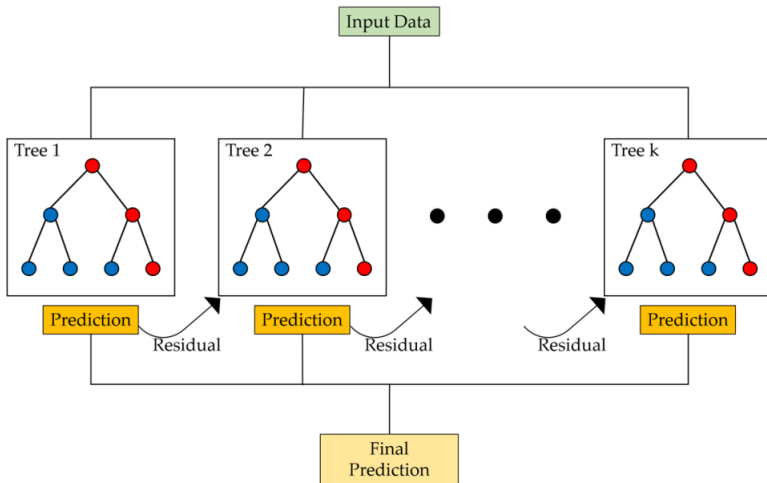
Hiperparâmetros do Random Forest

- **Número de árvores (M):**
 - Quanto maior, melhor (até certo ponto).
 - Valores como 500–1000 árvores geralmente são suficientes.
- **Número de atributos por split (m):**
 - Regra prática: $m \approx \sqrt{p}$ para classificação ($p = n^\circ$ de atributos).
 - Pode ser ajustado via validação cruzada.
- Outros:
 - Profundidade máxima da árvore.
 - Número mínimo de amostras por folha.

Boosting: ideia geral

- Objetivo: transformar muitos **classificadores fracos** em um **classificador forte**.
- Classificador fraco: desempenho só um pouco melhor que o chute.
- Intuição:
 - Treinamos um modelo.
 - Damos mais peso aos exemplos que ele errou.
 - Treinamos um segundo modelo focado nesses exemplos difíceis.
 - Repetimos o processo várias vezes.
- No final, combinamos os modelos em uma **combinação ponderada**.

Boosting



Bagging vs. Boosting

Bagging

- Modelos treinados em paralelo.
- Cada modelo vê uma amostra bootstrap dos dados.
- Objetivo principal: reduzir **variância**.
- Cada modelo tem o mesmo peso na combinação.

Boosting

- Modelos treinados em **sequência** (cascata).
- Cada novo modelo foca nos exemplos mal classificados.
- Objetivo principal: reduzir **viés**.
- Modelos recebem pesos diferentes na combinação final.

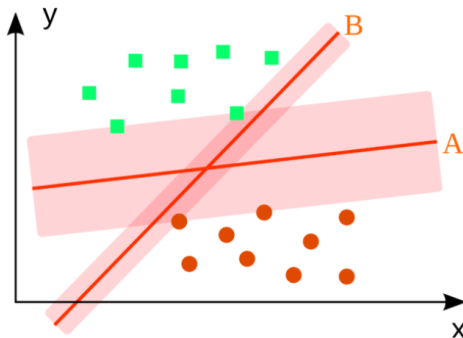
AdaBoost (Adaptive Boosting)

- Versão clássica de Boosting para classificação binária.
- Passos simplificados:
 - 1 Começamos com pesos iguais para todas as amostras.
 - 2 Treinamos um classificador fraco (ex.: árvore rasa).
 - 3 Calculamos o erro ponderado e o peso α_t desse classificador.
 - 4 Aumentamos o peso das amostras que foram classificadas **erradas**, reduzindo o das corretas.
 - 5 Repetimos por T iterações.
- Predição final:

$$F(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

onde h_t é o t -ésimo classificador fraco.

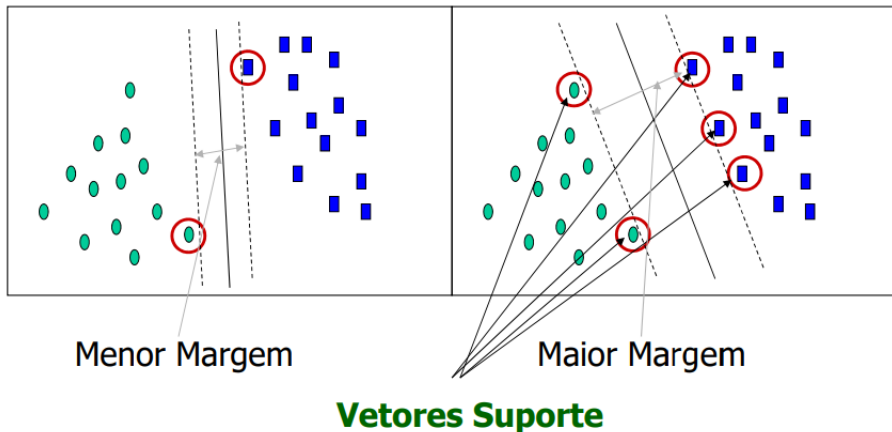
Support Vector Machines (SVM)



Motivação para SVM

- Em muitos problemas, queremos separar duas classes com um **hiperplano**.
- Mas nem todo hiperplano é igualmente bom.
- Ideia da SVM:
 - Escolher o hiperplano que **maximiza a margem**, ou seja, a distância até os exemplos mais próximos de cada classe.
 - Esses exemplos mais próximos são os **vetores de suporte**.
- Analogia: esticar ao máximo uma corda entre duas nuvens de pontos sem encostar em nenhuma delas.

Support Vector Machines (SVM)



SVM linear: margem máxima

- Para dados linearmente separáveis, procuramos um hiperplano

$$f(x) = \mathbf{w}^\top \mathbf{x} + b = 0$$

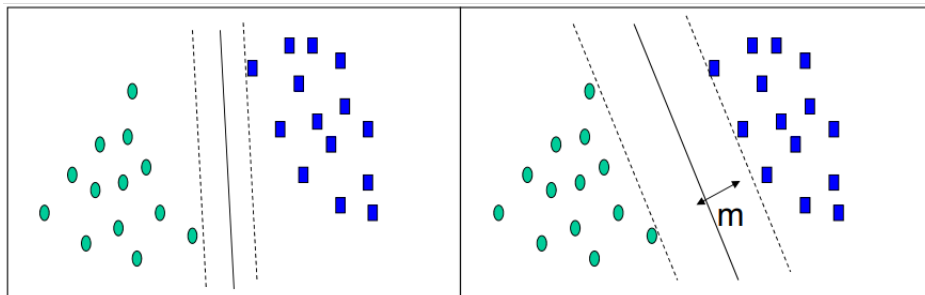
que separa as duas classes.

- A SVM resolve o seguinte problema de otimização:

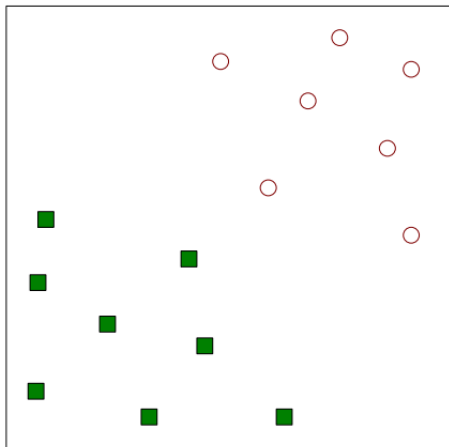
$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{sujeito a } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \forall i$$

- Isso corresponde a maximizar a margem entre as duas classes.
- Na prática, usamos versões **soft-margin** que permitem alguns erros, controlados por um hiperparâmetro C (trade-off entre margem larga e poucos erros).

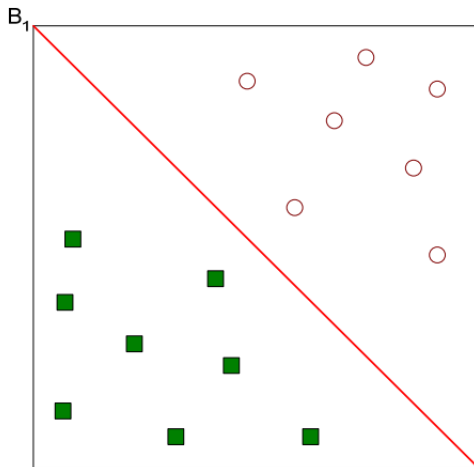
SVM - Dados separados linearmente



Encontrar um hiperplano linear (limite de decisão) capaz de separar os dados

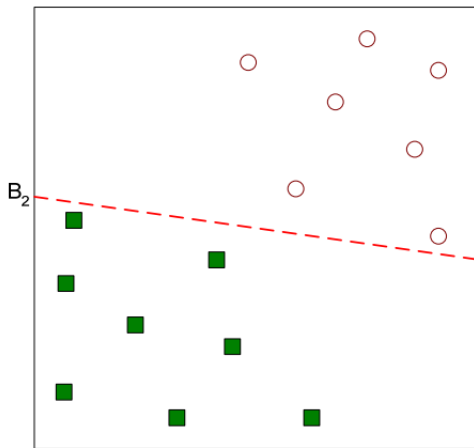


Uma possível solução

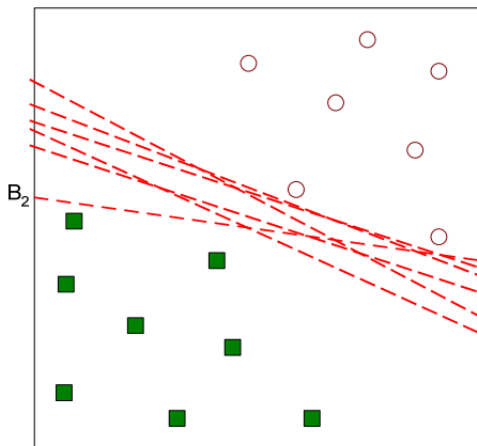


.

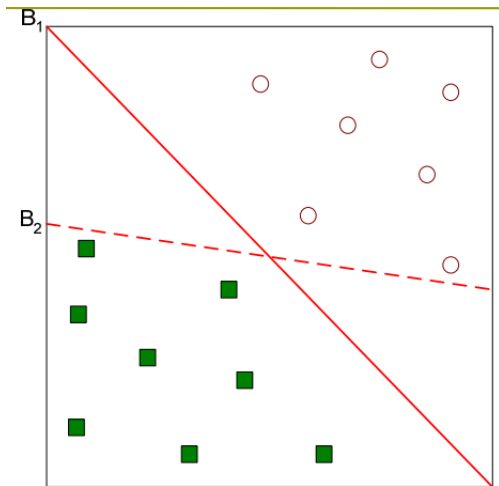
Outra possível solução



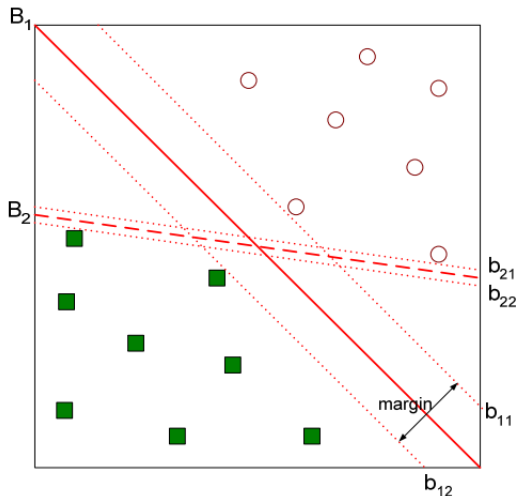
Outras possíveis soluções



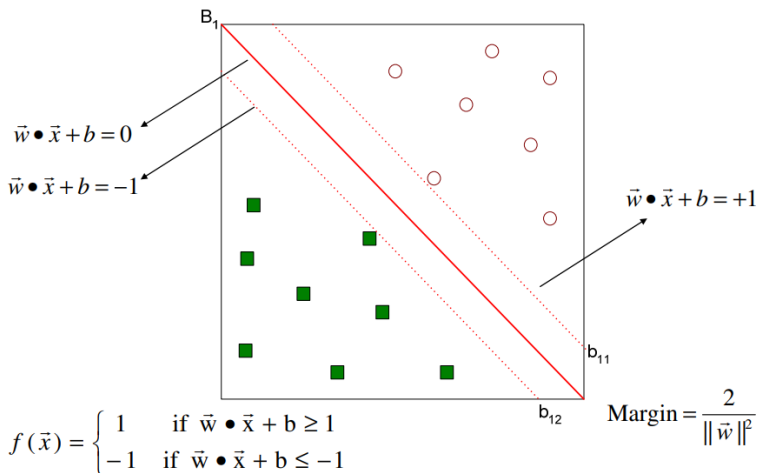
Qual é o melhor hiperplano? B1 ou B2?



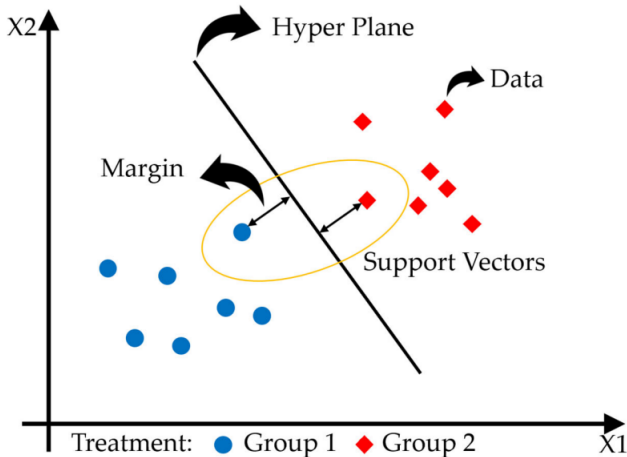
encontrar um hiperplano que maximize a margem
→ B1 é melhor do que B2.



Support Vector Machine



Support Vector Machine



Kernels: SVM não lineares

- Muitos problemas não são linearmente separáveis no espaço original.
- Ideia dos **kernels**:
 - Mapear os dados para um espaço de dimensão maior onde a separação é linear.
 - Em vez de calcular o mapeamento explicitamente, usamos uma função kernel $K(x_i, x_j)$ que calcula o produto interno nesse novo espaço.
- Exemplos de kernels:
 - Linear: $K(x_i, x_j) = x_i^\top x_j$.
 - Polinomial.
 - RBF (gaussiano).
- Importante: a complexidade do classificador depende do número de **vetores de suporte**, não da dimensionalidade do espaço.

Classificação multiclasse

- Até agora falamos em **duas classes** (positivo/negativo), mas na prática muitas vezes temos $C > 2$ classes.
- Ex.: tipos de uso do solo, diferentes espécies de plantas, vários tipos de poluente.
- Estratégias gerais:
 - **Algoritmos nativamente multiclasse:** árvores, Random Forest, redes neurais.
 - **Decomposição em problemas binários**, por exemplo:
 - **One-vs-All (OvA):** um classificador por classe vs. todas as outras.
 - **One-vs-One (OvO):** um classificador para cada par de classes.
- A escolha da estratégia impacta tempo de treino e interpretação.

kNN: k-Nearest Neighbors

- **Lazy learner:** não há fase de treinamento explícita.
- Para classificar um novo ponto:
 - 1 Calcula a distância até todos os pontos de treino.
 - 2 Seleciona os k vizinhos mais próximos.
 - 3 A classe é a **mais frequente** entre esses vizinhos (ou média, no caso de regressão).
- Medida de distância típica: Euclidiana, mas outras são possíveis.
- Hiperparâmetro k :
 - k pequeno $\Rightarrow$ modelo sensível a ruído (overfitting).
 - k grande $\Rightarrow$ modelo muito suave (underfitting).

Validação cruzada

- Objetivo: estimar a capacidade de **generalização** de um modelo.
- **Holdout:**
 - Divide os dados em treino (p.ex. 2/3) e teste (1/3).
 - Simples, mas a avaliação depende do acaso do split.
- **k-fold Cross-Validation:**
 - Divide em k partes.
 - Treina em $k - 1$ partes e testa na parte restante, repetindo k vezes.
 - Média das métricas ao longo dos folds.
- Usamos CV tanto para:
 - Avaliar modelos (RF, SVM, kNN, etc.).
 - Escolher hiperparâmetros (grid search, random search).

Ajuste de hiperparâmetros

- Exemplo: escolher o melhor k no kNN, C e kernel na SVM, n° de árvores no Random Forest.
- Procedimento típico:
 - 1 Definir um conjunto de valores candidatos para cada hiperparâmetro.
 - 2 Para cada combinação:
 - Aplicar validação cruzada no conjunto de treino.
 - Calcular a métrica média (acurácia, F1, AUC, etc.).
 - 3 Escolher a combinação com melhor desempenho médio.
 - 4 Treinar um modelo final usando todos os dados de treino com esses hiperparâmetros.
- Cuidado com **overfitting** nos hiperparâmetros: manter um conjunto de teste separado quando possível.

Resumo da aula

- **Comitês de classificadores** combinam vários modelos para melhorar robustez e desempenho.
- **Bagging** reduz variância; **Random Forest** aplica bagging + seleção aleatória de atributos.
- **Boosting** reduz viés, focando em exemplos difíceis; **AdaBoost** é um exemplo clássico.
- **SVM** busca hiperplanos de margem máxima e pode usar kernels para separações não lineares.
- Vimos ainda:
 - Estratégias para **classificação multiclasse** (OvA, OvO).
 - **kNN** como exemplo de classificador baseado em instância.
 - **Validação cruzada** e **ajuste de hiperparâmetros**.